

Blockchain-Based E-Voting System for Enhanced Electoral Integrity

Major Project Report Submitted in Partial Fulfilment of the Requirements for the Degree
of

Bachelor of Engineering *in* Information Technology

Submitted by

Bhagat Singh Kankarwal: (Roll No. 22UITE5036)

Devanshu Tiwari: (Roll No. 22UITE5037)

Under the Supervision of

Dr. Simran Choudhary
Professor



Department of Computer Science and Engineering
MBM University, Jodhpur
May, 2025



Department of Computer Science & Engineering

MBM University, Ratanada, Jodhpur, Rajasthan, India -342011

CERTIFICATE

This is to certify that the work contained in this report entitled “**Blockchain-Based E-Voting System for Enhanced Electoral Integrity**” is submitted by the group members Mr. Bhagat Singh Kankarwal (Roll. No: 22UITE5036) and Mr. Devanshu Tiwari, (Roll. No: 22UITE5037) to the Department of Computer Science & Engineering, MBM University, Jodhpur, for the partial fulfilment of the requirements for the degree of **Bachelor of Engineering in Information Technology**.

They have carried out their work under my supervision. This work has not been submitted else-where for the award of any other degree or diploma.

The project work in our opinion, has reached the standard fulfilling of the requirements for the degree of Bachelor of Engineering in accordance with the regulations of the University.

Dr. Simran Choudhary
Professor
(Supervisor)
Dept. of Computer Science & Engg.

Dr. Shrwan Ram
(Head of Department)
Dept. of Computer Science & Engg.

DECLARATION

We, **Bhagat Singh Kankarwal** and **Devanshu Tiwari**, IV Year, hereby declare that this project titled “**Blockchain-Based E-Voting System for Enhanced Electoral Integrity**” is a record of original work done by us under the supervision and guidance of **Dr. Simran Choudhary, Professor, CSE**.

We, further certify that this work has not formed the basis for the award of the Degree/Diploma/Associateship/Fellowship or similar recognition to any candidate of any university and no part of this report is reproduced as it is from any other source without appropriate reference and permission.

Bhagat Singh Kankarwal (22UITE5036)

Devanshu Tiwari (22UITE5037)

ACKNOWLEDGEMENT

We would like to express our heartfelt appreciation to everyone who contributed to the completion of our college project on "**Blockchain-Based E-Voting System for Enhanced Electoral Integrity**". We are grateful for the support and assistance provided by various individuals and institutions.

We especially want to thank our project supervisor, Dr. Simran Choudhary, Professor, CSE. Her expert guidance and unwavering support throughout the project were instrumental in shaping its direction and ensuring its success. Her insights and knowledge in the field were invaluable to us.

We also acknowledge the support of Dr. Shrwan Ram, Head of the Department, CSE, for providing us with the necessary resources and fostering a conducive environment for research and learning. The department's commitment to academic excellence played a significant role in enabling us to pursue this project.

We would like to extend our sincere gratitude to the researchers and scholars in the field of Blockchain and Homomorphic Encryption whose pioneering work laid the foundation for our endeavours. Their dedication to advancing this transformative technology has opened doors to new possibilities for data privacy and security.

Finally, we express our heartfelt thanks to all the individuals and institutions who contributed to the success of this project, whether through direct involvement or by providing a supportive environment. Your contributions are deeply appreciated and have made this journey a truly rewarding experience.

ABSTRACT

Our project, BlockVote, addresses the critical need for secure, transparent, and auditable electronic voting systems. It proposes a novel solution leveraging the immutability of blockchain technology combined with the privacy-preserving capabilities of Homomorphic Encryption (HE) and Zero Knowledge Proofs to ensure voter confidentiality and election integrity. This system features a user-friendly web interface for voters and a secure administrative panel for election officials.

Through the BlockVote platform, registered voters can cast their ballots digitally. Crucially, each vote is encrypted on the client-side using homomorphic encryption scheme before being transmitted to the Hyperledger Fabric blockchain. This ensures that individual vote contents remain confidential even while stored on the distributed ledger and during the tallying process, mitigating risks of coercion and vote tampering. The administrative panel allows authorized officials to manage election parameters, oversee the voting process, and initiate the secure tallying of encrypted votes.

Homomorphic Encryption is the cornerstone of Block Vote privacy mechanism, enabling computations to be performed directly on encrypted data without requiring decryption of individual ballots. This eliminates vulnerabilities associated with traditional systems where votes might be exposed during counting. The election's public and private keys for the encryption are managed securely, with the private key used only at the final stage to decrypt the aggregated tally, revealing the election outcome without compromising individual voter choices. User authentication and identity management are robustly handled by SuperTokens, ensuring that only eligible and verified voters can participate.

The Hyperledger Fabric blockchain ensures transparent, tamper-proof recording of encrypted ballots and election transactions through distributed ledger technology, enhancing election auditability and verifiability. By integrating blockchain, homomorphic encryption, zero-knowledge proofs, and SuperTokens secure authentication, BlockVote presents a comprehensive solution for modernizing electoral systems. This platform demonstrates the potential for building trustworthy, secure, and user-centric e-voting systems that strengthen democratic processes and public confidence in elections.

Contents

1. Introduction	1
1.1. Project Description	1
1.2. Motivation	2
1.3. Need of the Project	3
2. Requirement Modelling	4
2.1. Functional Requirements	4
2.1.1 User Management & Authentication	4
2.1.2 Election Management	5
2.1.3 Voting Process	5
2.1.4 Results Management & Verification	5
2.2. Non-Functional Requirements	6
2.2.1 Performance	6
2.2.2 Security	7
2.2.3 Usability	7
2.2.4 Scalability	7
2.3. Deployment Environment	7
3. System Design	8
3.1 System Architecture	8
3.1.1 Blockchain Layer	8
3.1.2 Backend Layer	9
3.1.3 Frontend Layer	9

3.2	Module Decomposition	10
3.3	Database Design	11
3.4	Important Algorithms	12
4.	Project Implementation	14
4.1	Technology Stack	14
4.2	Code Implementation	14
4.2.1	Frontend Implementation	14
4.2.2	Backend Implementation	19
4.3	Ballot Encryption	21
4.4	Ballot Decryption	22
4.5	Project Installation and Execution	23
5.	Conclusion & Future Work	25
5.1	Conclusion	25
5.2	Future Work	25
	References	27

List of Figures

1.1 Home Page	2
2.1 User Registration - UML Sequence Diagram	6
3.1 System Architecture	10
3.2 State Objects Diagram	11
3.3 ZKP Voting Flow	13
3.4 Data Flow Diagram of Application	13
4.1 Identity Creation for Voter	17
4.2 Voter Dashboard	17
4.3 Voter Dashboard after Voting	18
4.4 Admin Dashboard	18
4.5 Project Directory Structure	23

Chapter 1

Introduction

The electoral process stands at a crossroads, balancing the fundamental need for transparent, verifiable elections with the critical responsibility of safeguarding voter privacy and ballot secrecy. Traditional voting mechanisms often face challenges in ensuring end-to-end security and privacy, potentially exposing vulnerabilities to tampering or compromising voter anonymity during tabulation[1].

This project, BlockVote, introduces a paradigm shift by harnessing the power of blockchain technology, specifically Hyperledger Fabric, in conjunction with Homomorphic Encryption, to create a secure and privacy-preserving electronic voting platform. By enabling computations, such as vote tallying, directly on encrypted data, this approach significantly reduces the risk of exposing individual votes, ensuring that voter choices remain confidential throughout the electoral process. This project delves into the design and implementation of such a system, exploring its potential to revolutionize election integrity and voter confidence, paving the way for a more transparent, auditable, and trustworthy democratic framework.

1.1 Project Description

Our project is a decentralized electronic voting platform built on Hyperledger Fabric blockchain to ensure secure, transparent, and tamper-proof elections. The system addresses traditional voting vulnerabilities through immutable vote recording and end-to-end verifiability. The system facilitates the entire election lifecycle, from user registration and authentication, through secure vote casting and immutable recording on the blockchain, to privacy-preserving vote tallying and result announcement.

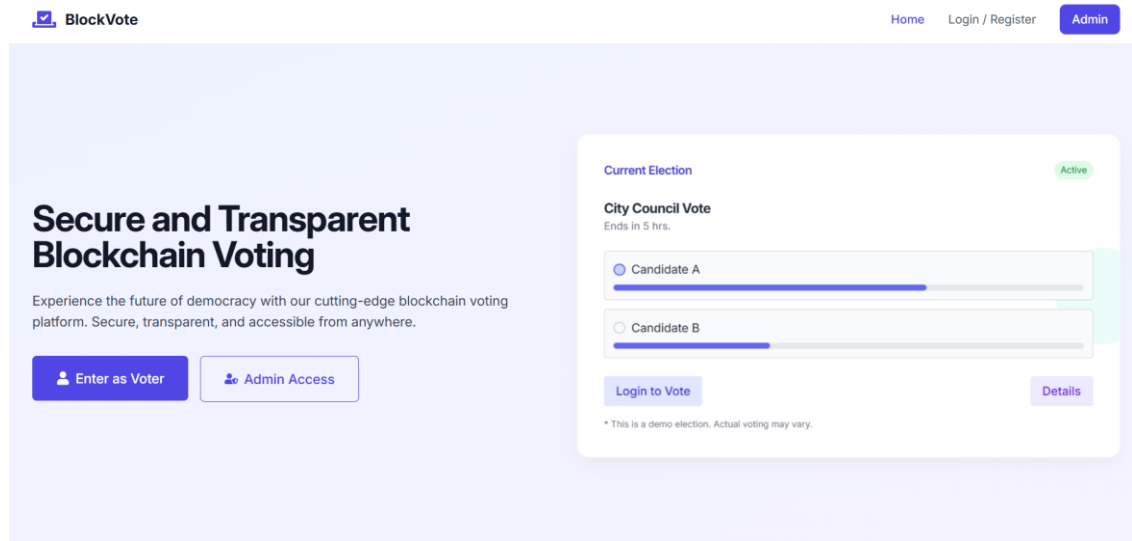


Fig 1.1 Home Page

1.2 Motivation

The motivation for the project stems from the inherent challenges in traditional voting systems regarding transparency, security, and the safeguarding of voter privacy. Current electoral processes, despite measures like EVMs and VVPATs, often face scrutiny and distrust[2]. BlockVote aims to address these concerns by leveraging Hyperledger Fabric blockchain technology and homomorphic encryption to create a voting platform that is not only secure and tamper-proof but also ensures voter anonymity.

Key drivers include:

- **Enhancing Trust:** Leveraging blockchain's transparency and tamper-proof nature to build public confidence in electoral processes[3].
- **Improving Security & Privacy:** Tackles issues of trust, potential tampering, privacy breaches in traditional systems, and physical vulnerabilities like booth capturing.
- **Exploring Enterprise Blockchain:** Utilizing Hyperledger Fabric's capabilities for controlled, governance-friendly institutional voting solutions.

1.3 Need of the Project

BlockVote addresses critical needs in modern electoral systems:

- **Increased Transparency & Auditability:** Providing a verifiable and immutable ledger of votes, unlike opaque traditional or electronic systems[4].
- **Enhanced Security Against Fraud:** Offering a decentralized, cryptographically secured platform less susceptible to tampering compared to centralized databases.
- **Stronger Voter Privacy:** Implementing Homomorphic Encryption to tally votes without decryption, ensuring individual choice secrecy[5].
- **Modern, Efficient Solutions:** Demonstrating a pathway towards more streamlined and potentially cost-effective institutional voting using open-source technologies.
- **Countering Misinformation:** Building demonstrably fair systems to combat scepticism about election integrity.

Chapter 2

Requirement Modelling

For this project, a detailed analysis of requirements has been conducted to guide the design, development, and testing of a secure, transparent, and privacy-preserving blockchain-based electronic voting platform.

2.1 Functional Requirements

In the context of this project, functional requirements specify the core operations and behaviour of the system, detailing interactions from user management to election processes and administrative oversight.

2.1.1 User Management & Authentication

2.1.1.1 Voter Registration & Login

- The system shall allow new users to register as voters.
- Registered voters shall be able to log in to the system.
- The system shall integrate with Hyperledger Fabric CA for user identity management upon registration.

2.1.1.2 Admin Login

- The system shall provide a separate login mechanism for administrators.

2.1.1.3 Identity Management

- Fabric CA integration for X.509 certificate issuance.
- Mapping between SuperTokens identities and blockchain identities.

2.1.1.4 Role-Based Access Control

- The system shall differentiate functionalities and access based on user roles (Voter, Administrator).

2.1.2 Election Management

2.1.2.1 Election Creation and Configuration

- Administrators shall be able to create new elections, defining parameters such as name, description, start/end dates, and candidates.

2.1.2.2 View Elections

- Administrators shall be able to view a list of all elections.
- Administrators shall be able to view details of specific elections.

2.1.2.3 Election Monitoring & Management

- Administrators shall be able to monitor the status of elections.
- Administrators shall be able to manage the lifecycle of an election.

2.1.3 Voting Process

2.1.3.1. View Active Elections

- Voters shall be able to view a list of currently active elections they are eligible for.

2.1.3.2. Secure Vote Casting

- The system shall ensure votes are encrypted (using homomorphic encryption) before being submitted to the blockchain.

2.1.4 Results Management and Verification

2.1.4.1 View Election Results

- Once an election is closed, both administrators and voters (publicly) shall be able to view the election results.
- Results should include individual vote count.

2.1.4.2 Secure & Verifiable Tallying

- The system shall tally votes securely, preserving voter privacy through homomorphic encryption.
- The system should allow for the verification of ballots if required.

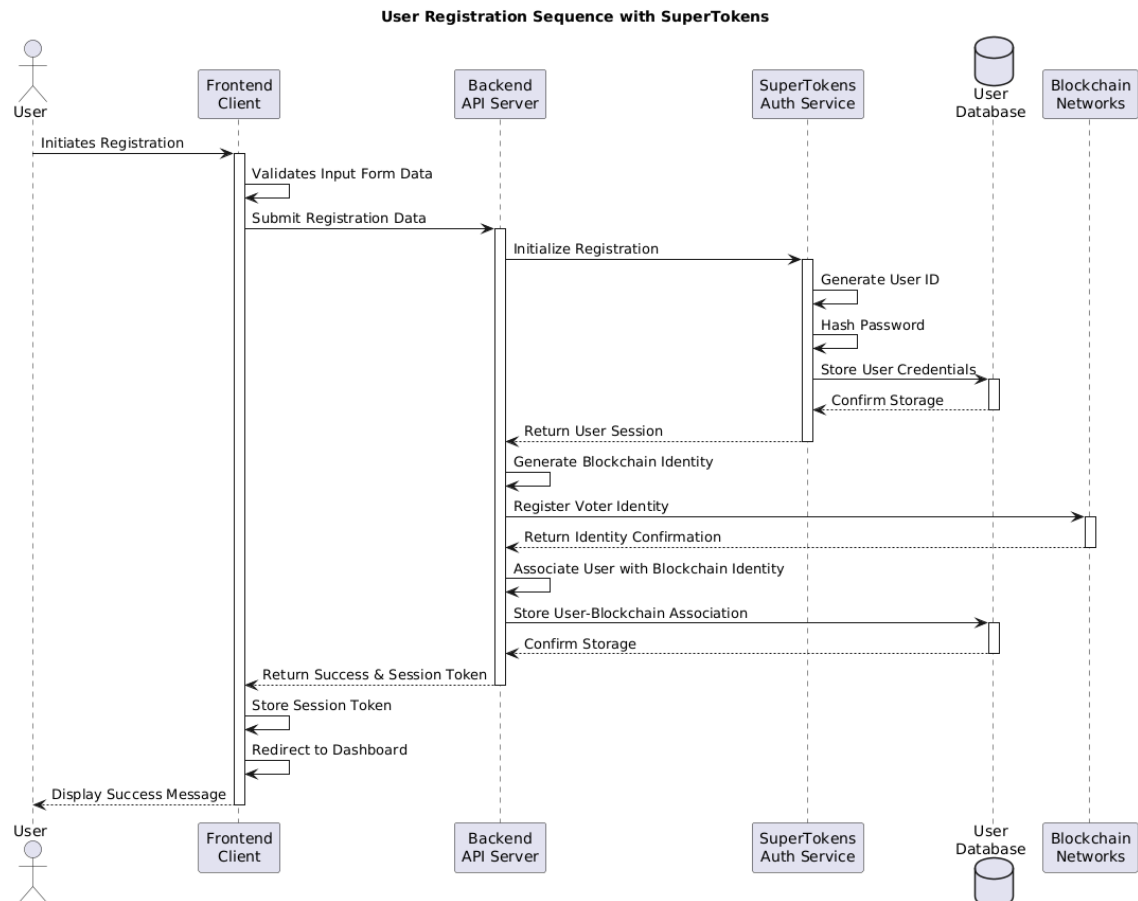


Fig 2.1 User Registration - UML Sequence Diagram

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system. They describe how the system should behave.

2.2.1 Performance

- The system shall respond to user actions and requests within a reasonable time frame.

- Homomorphic encryption and decryption operations for vote tallying should not introduce undue delays, with results.

2.2.2 Security

- All communication between the client (frontend) and backend, and between the backend and the blockchain network, shall be encrypted using TLS/HTTPS.
- Homomorphic Encryption for vote privacy and Zero-Knowledge Proofs for verification[6].
- All critical administrative actions (e.g., election creation, ledger initialization) and voting transactions shall be logged on the blockchain for audit purposes.

2.2.3 Usability

- The voting process shall be intuitive, requiring minimal instruction for a first-time user.
- Voters shall receive clear confirmation upon successfully casting their vote.
- The admin dashboard shall provide a clear and efficient interface for creating, managing, and monitoring elections.
- Cross-device compatibility and modern UI components.

2.2.4 Scalability

- Modular architecture with Docker containerization for horizontal scaling.
- Multi-organization network support with extensible channel configuration.
- RAFT consensus algorithm for high throughput and fault tolerance.

2.3 Deployment Environment

The deployment environment plays a critical role in ensuring the system's accessibility, performance, and security. For this blockchain-based e-voting project, a robust and distributed environment, likely leveraging cloud infrastructure, will provide the foundation for optimal functionality and trustworthiness. The inherent security features of Hyperledger Fabric, combined with homomorphic encryption for vote privacy, allow for a distributed deployment model where components can reside on trusted cloud servers.

Chapter 3

System Design

The complex architecture and design ideas that support the blockchain-based voting system are revealed in this chapter. This section also highlights the key algorithms that drive the security and operation of the system.

3.1 System Architecture

The BlockVote platform implements a comprehensive three-tier architecture that seamlessly combines blockchain technology with traditional web components to deliver a secure, scalable, and user-friendly voting solution:

3.1.1 Blockchain Layer : The foundational layer of the system is a permissioned blockchain network implemented using Hyperledger Fabric[7].

- **Network Configuration:** The network typically consists of multiple peer nodes, an ordering service to ensure transactional consensus, and Certificate Authorities (CAs) for managing digital identities. A channel, named ‘mychannel’, isolates transactions and ledger data specific to the voting application.
- **Chaincode:** A smart contract, developed in TypeScript, governs the core logic of the electoral process. The smart contract ensures that all voting rules are enforced and that data recorded on the ledger is immutable and auditable.
- **Identity Management:** Hyperledger Fabric CA is utilized for issuing and managing cryptographic identities for participants (voters and administrators), ensuring that only authorized entities can interact with the network.

3.1.2 Backend Layer : The backend application serves as an intermediary layer, facilitating communication between the frontend and the Hyperledger Fabric network, and managing business logic not directly encoded in the smart contract.

- **Technology Stack:** It is developed using Node.js and the Express.js framework, providing a RESTful API for client interactions.
- **Blockchain Interaction:** The backend utilizes the Hyperledger Fabric Gateway SDK to connect to the blockchain network, invoke smart contract functions, and query ledger data. Communication with Fabric peers is handled via gRPC.
- **Authentication and Authorization:** User authentication is managed through SuperTokens, which handles user registration, login, and session management. For administrative functions, JSON Web Tokens (JWT) are employed to secure access to privileged operations. The backend maps authenticated users to their respective Fabric identities.
- **Homomorphic Encryption Services:** A critical function of the backend is to manage the homomorphic encryption scheme. This includes handling the generation or retrieval of encryption keys and ensuring that votes are appropriately encrypted before submission to the blockchain and decrypted (for tallying) under controlled, secure conditions.

3.1.3 Frontend Layer : The frontend provides an accessible and intuitive interface for voters and administrators to interact with the BlockVote system.

- **Technology Stack:** It is a single-page application (SPA) built with React and Vite, utilizing TailwindCSS for styling and Framer Motion for user interface animations.
- **State Management and Routing:** Client-side routing is handled by React Router, and component state is managed within React, ensuring a responsive user experience.
- **API Communication:** The frontend communicates with the backend via RESTful API

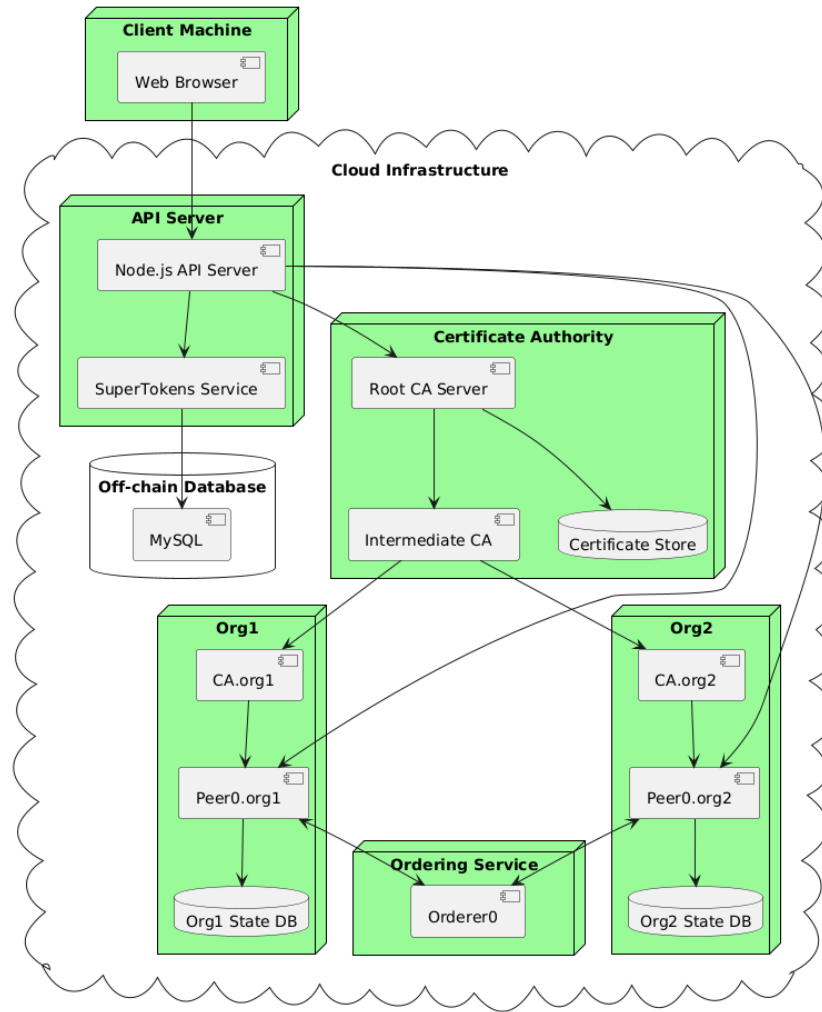


Fig. 3.1 System Architecture

3.2 Module Decomposition

The project is divided into distinct modules, each responsible for specific functionalities:

- **Frontend Module (React):** Contains **VotingDashboard** for voters and **AdminDashboard** for administrators.
- **API Module (Node.js):** Uses **voting.router.js** to handle requests for election management, voting, and results.
- **Authentication Module (SuperTokens):** Manages user identity and integrates with Fabric CA to map web identities to blockchain identities.
- **Smart Contract Module (TypeScript):** The **VotingContract** on Hyperledger Fabric manages Election data, Candidate information, encrypted **PrivateVote** ballots, and **EncryptionKeys** for homomorphic encryption.

3.3 Database Design

- **Blockchain State DB (On-Chain):** A distributed, immutable ledger. This is the core repository for all critical election-related data. Stores core voting records like election metadata (*ELECTION_<electionID>*) and encrypted ballots (*VOTE_<electionID>_<voterID>*).
- **MySQL Database (Off-Chain):** Managed by SuperTokens to handle user account, authentication and session data.
- **Local Storage (Server-Side):** The backend also maintains a local file system to store registered users id, admin certificates and admin tokens.
- **Local Storage (Client-Side):** The frontend application uses the browser's localStorage to store user-specific blockchain identity credentials. It also caches the user session ID's generated by SuperTokens service to allow secure access to system.

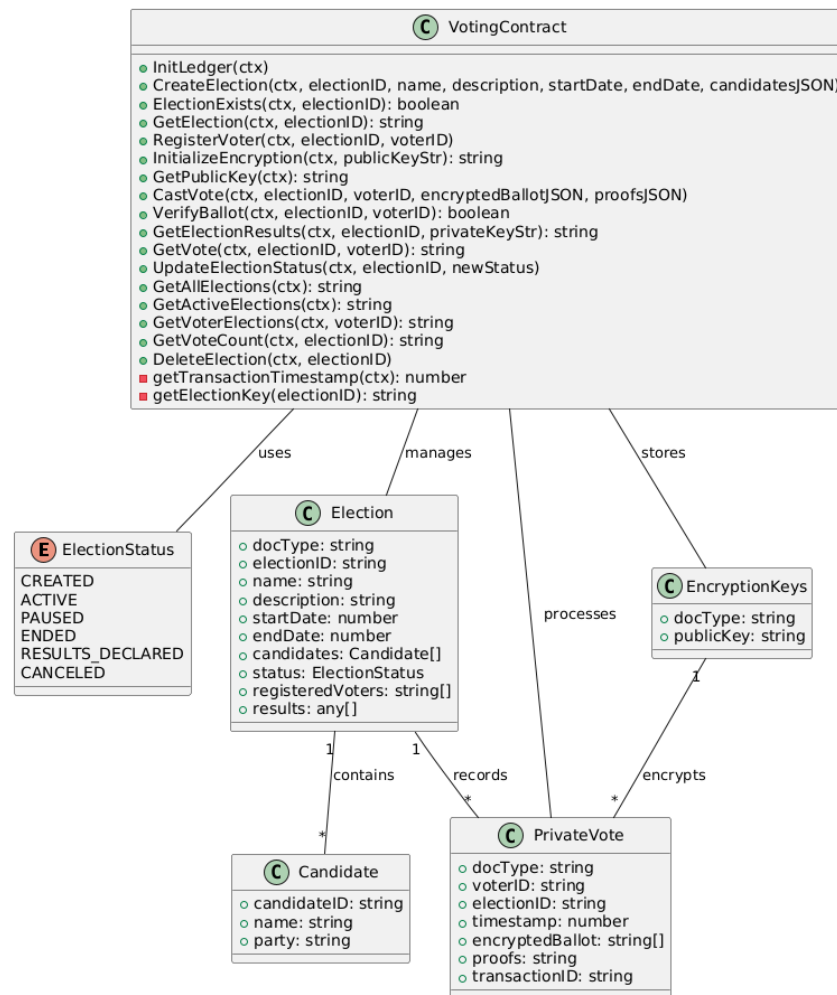


Fig. 3.2 State Objects Diagram

3.4 Important Algorithms

3.4.1 Homomorphic Encryption (Paillier Cryptosystem)

BlockVote uses partial homomorphic encryption, specifically the Paillier cryptosystem, to ensure that votes remain private and secure on the blockchain. This allows encrypted votes to be tallied without decrypting individual ballots, preserving voter privacy. The cryptographic operations are handled by the `mind-paillier-voting-sdk` library.

The process is as follows:

1. **Encryption:** Votes are encrypted before being stored on the blockchain ledger.
2. **Homomorphic Tallying:** The blockchain smart contract adds these encrypted votes together, resulting in an encrypted total.
3. **Decryption:** Only the final encrypted sum is decrypted with a private key to reveal the election outcome, ensuring individual votes are never exposed.

3.4.2 Zero-Knowledge Proofs (ZKP)

To ensure that each encrypted ballot is valid BlockVote uses zero-knowledge proofs. These proofs are generated and submitted along with the encrypted ballot, and are verified on-chain using the Verifier from the `mind-paillier-voting SDK`[8]. This prevents invalid or fraudulent votes from being counted, while maintaining privacy.

3.4.3 Raft consensus mechanism

The Raft consensus mechanism[9] is a distributed protocol designed to achieve consensus among multiple nodes in a fault-tolerant way.

- **Leader Election:** One leader handles request; if it fails, a new leader is elected.
- **Log Replication:** The leader appends and replicates logs to followers; entries are committed once a majority confirms.
- **Safety:** Ensures all nodes maintain the same ordered log entries.
- **Term-Based Voting:** Each term elects a single leader, avoiding conflicts and ensuring consistency.

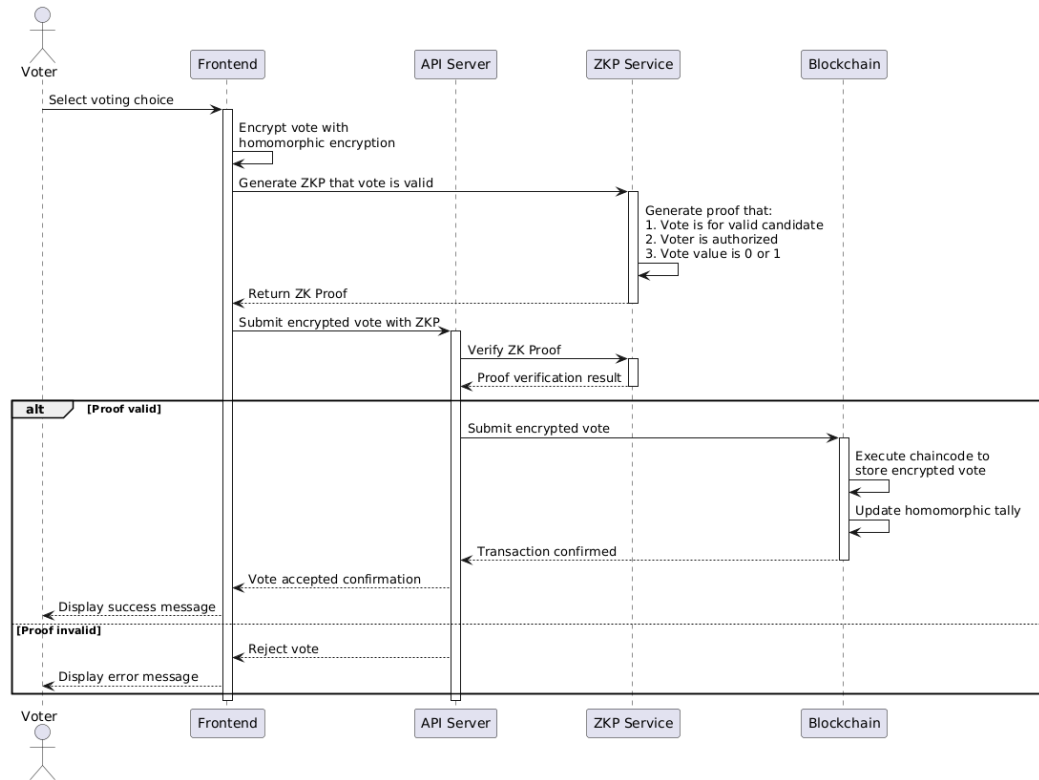


Fig 3.3 ZKP Voting Flow

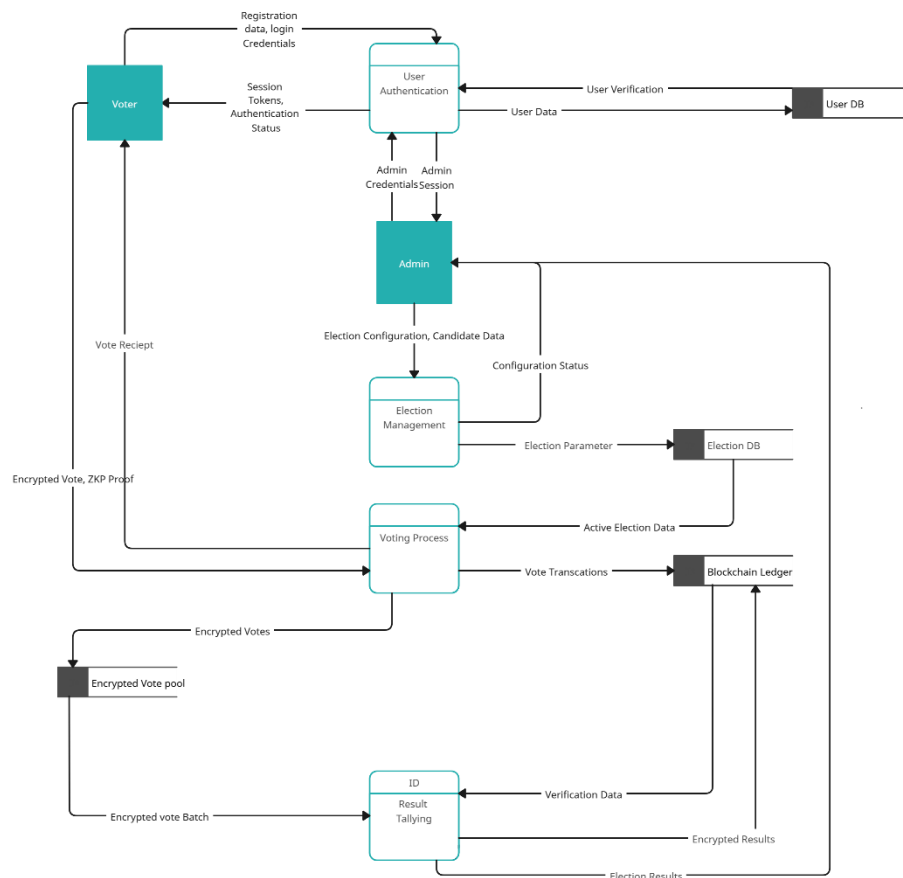


Fig 3.4 Data Flow Diagram of Application

Chapter 4

Project Implementation

4.1 Technology Stack

Frontend: React, TailwindCSS, Framer Motion, Chart.js, React Icons, Heroicons, Axios, React Router, Sonner, SuperTokens for authentication.

Backend: Node.js, Express.js, Hyperledger Fabric Node SDK, SuperTokens, mind-paillier-voting sdk, gRPC.

Blockchain: Hyperledger Fabric network with TypeScript-based smart contracts.

4.2 Code Implementation

4.2.1 Frontend Implementation

- **UI Components**

These are the reusable building blocks for the user interface. Components encapsulate specific UI elements and their logic, promoting modularity and consistency across the application. They range from simple elements like buttons to more complex structures like modals or layout sections.

```
import { motion } from 'framer-motion';  
function Button({  
  children,  
  variant = 'primary',  
  size = 'md',  
  fullWidth = false,  
  isLoading = false,  
  icon,
```

```

    ...props
  }) {
    const variants = {
      primary: 'bg-primary-600 text-white hover:bg-primary-700',
      secondary: 'bg-secondary-600 text-white hover:bg-secondary-700',
      outline: 'bg-transparent border border-primary-600 text-primary-600
hover:bg-primary-50'
    };
    const sizes = {
      sm: 'text-sm px-4 py-2',
      md: 'px-6 py-3',
      lg: 'text-lg px-8 py-4'
    };

    return (
      // Implementation
    );
  }
export default Button;

```

- **API Service**

This is responsible for all communication with the backend server. It typically involves a centralized way to make HTTP requests (e.g., GET, POST), handle responses, manage API base URLs, and potentially include error handling or request/response transformations.

```

import axios from 'axios';
import Session from 'supertokens-auth-react/recipe/session';

// Create axios instance with base URL
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:9010',
  headers: {
    'Content-Type': 'application/json'
  },
  withCredentials: true
});

api.interceptors.request.use(
  // Implementation
);

api.interceptors.response.use(
  (response) => {
    return response;
  }
);

```

```

    },
    async (error) => {
      // Error handling
    });

export default api;

```

- **Routing Configuration**

It defines how users navigate through different parts of the application. It maps URL paths to specific React components (pages or views), enabling a single-page application (SPA) experience.

```

function App() {
  return (
    <SuperTokensWrapper>
      <BrowserRouter>
        <div className="min-h-screen flex flex-col">
          <Toaster position="top-right" richColors />
          <Layout>
            <Routes>
              <Route path="/" element={<Home />} />
              {getSuperTokensRoutesForReactRouterDom(reactRouterDom,
[EmailPasswordPreBuiltUI, EmailVerificationPreBuiltUI])}
              <Route path="/dashboard" element={
                <SessionAuth requireAuth={true}>
                  <VotingDashboard />
                </SessionAuth>
              } />
              <Route path="/admin" element={<AdminLogin />} />
              <Route path="/admin/dashboard" element={<AdminDashboard />} />
            </Routes>
          </Layout>
        </div>
      </BrowserRouter>
    </SuperTokensWrapper>
  );
}

```

- **Voter Dashboard**

The Voter Dashboard provides voters with a secure, privacy-focused interface to participate in blockchain-based elections.

Upon first access, users are guided through a blockchain identity setup process where they can either generate new credentials or upload previously saved ones, with the system storing these securely in the browser's local storage.

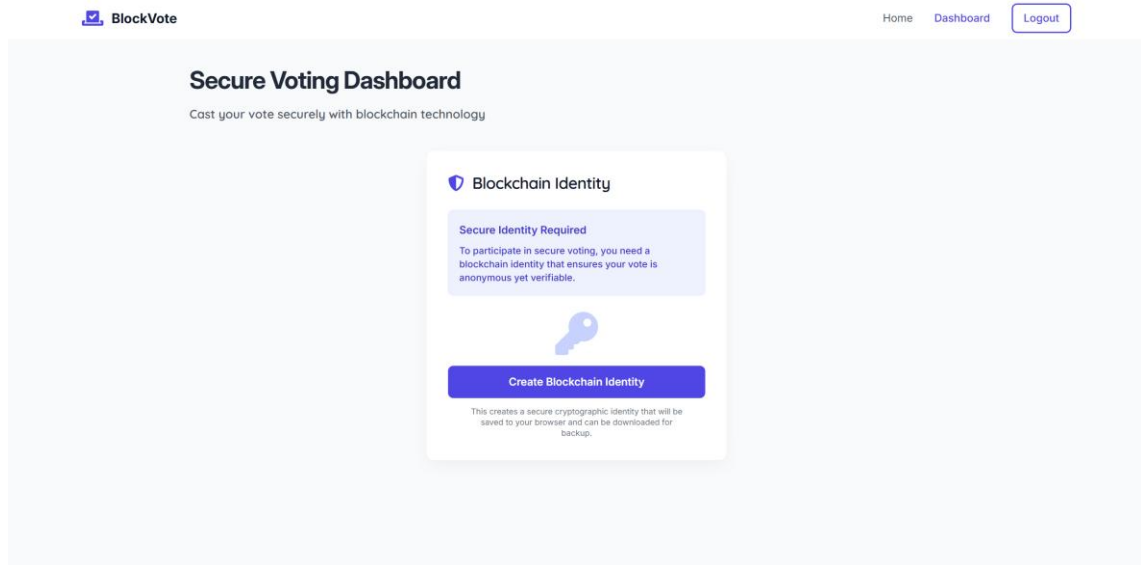


Fig. 4.1 Identity Creation for Voter

Once authenticated, voters can view all active elections, register for those they're eligible to participate in, and cast encrypted votes using homomorphic encryption.

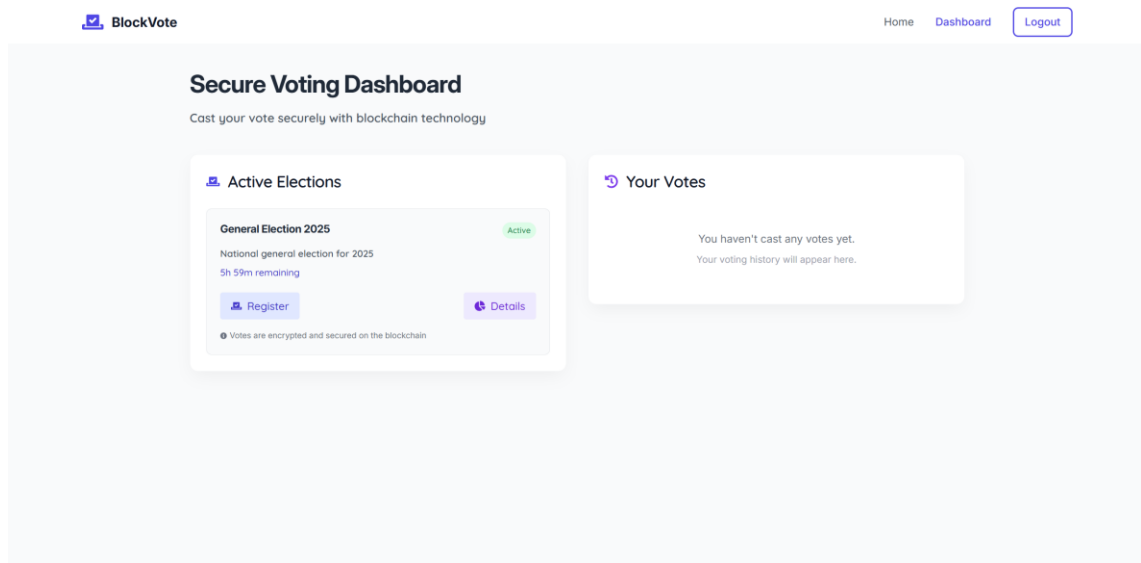


Fig. 4.2 Voter Dashboard

After voting, users can view their participation history, transaction IDs for verification purposes, and access results of completed elections.

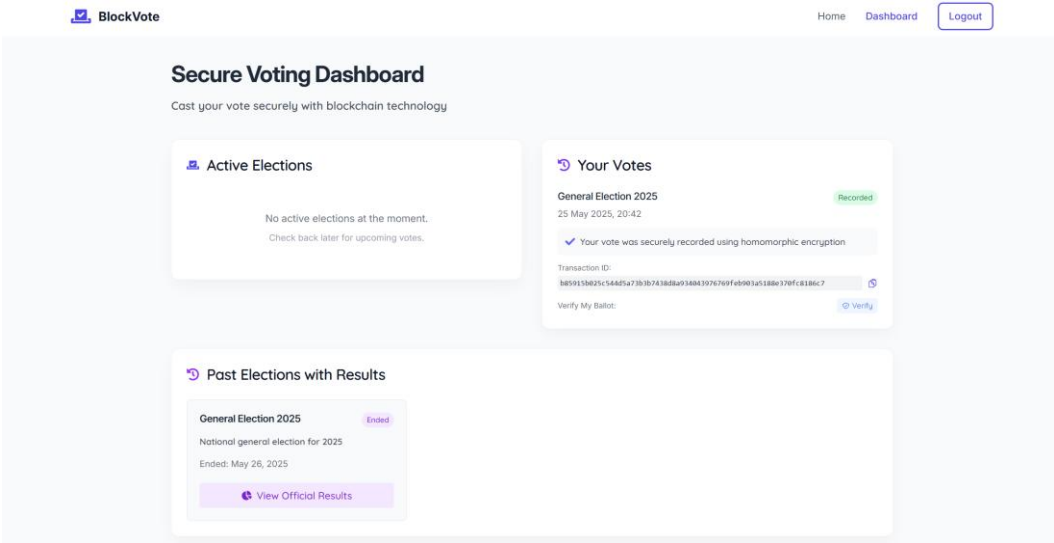


Fig. 4.3 Voter Dashboard after Voting

• Admin Dashboard

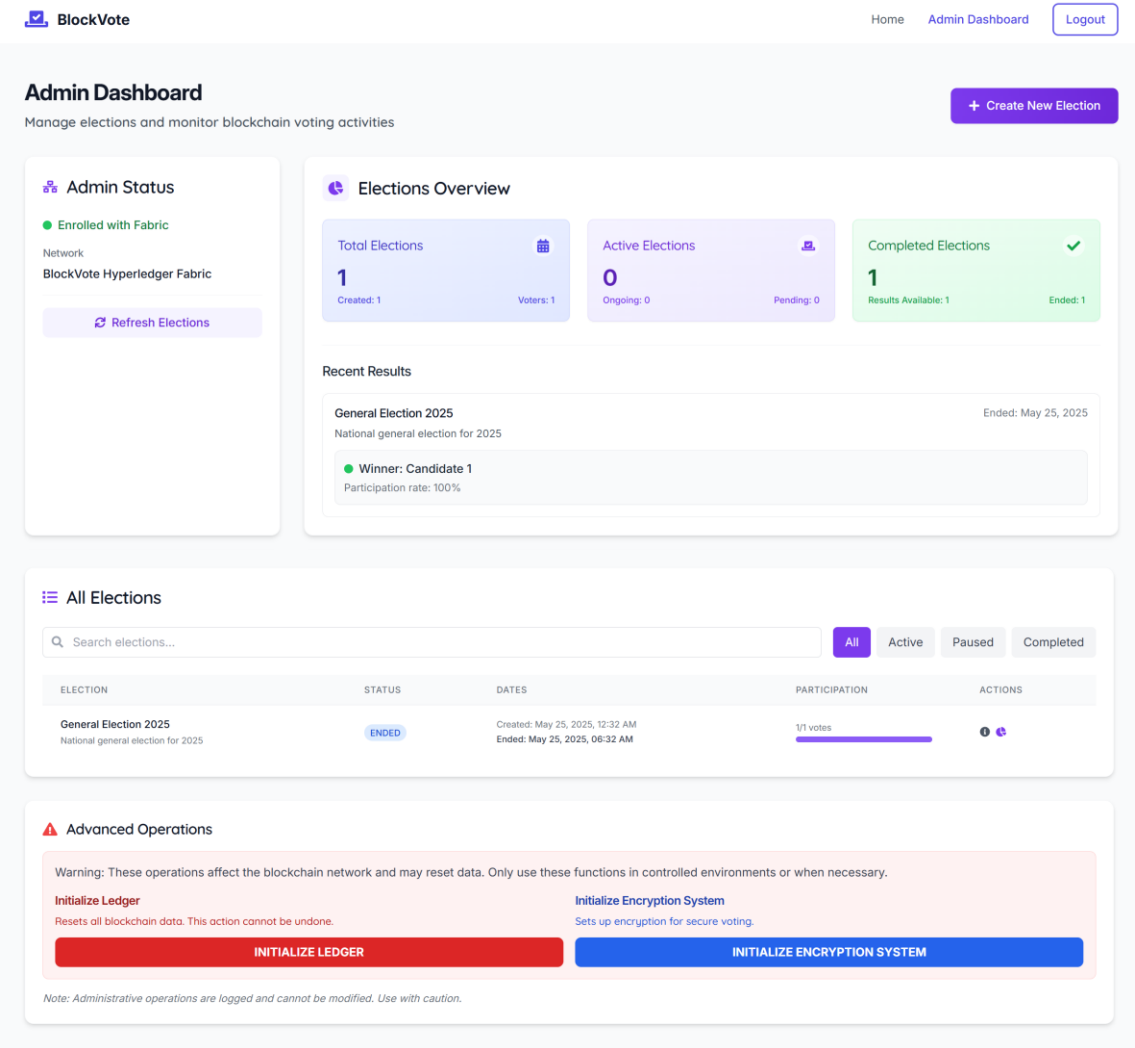


Fig. 4.4 Admin Dashboard

The Admin Dashboard serves as the command center for election management within the project, providing comprehensive controls for the entire electoral process.

Administrators can create, monitor, and manage elections through an intuitive interface that displays key metrics and election statuses at a glance. The dashboard enables admins to perform critical actions including creating new elections with customizable parameters, updating election statuses (activating, pausing, ending elections), viewing detailed election results with vote tallies, and managing voter registrations.

Advanced operations are available, such as initializing the blockchain ledger and the homomorphic encryption system necessary for secure voting. The interface implements robust security measures, requiring proper authentication and enrollment with the Fabric Certificate Authority before granting access to these privileged functions.

4.2.2 Backend Implementation

- **Server Configuration & Initialization**

This layer handles the Express server setup, middleware configuration, authentication integration, and blockchain connection initialization. It serves as the entry point for the application and coordinates the initialization of all other components.

```
// Initialize the blockchain connection
const connection = new Connection();
initBlockchainConnection(connection);

// Initialize the AuthRouter for Fabric functionality
const authRouter = new AuthRouter();
authRouter.routes(app, verifySession);

// Initialize the VotingRouter for blockchain voting functionality
const votingRouter = new VotingRouter();
votingRouter.routes(app, verifySession,
authRouter.verifyAdminToken.bind(authRouter));
```

- **Authentication & Identity Management**

This layer manages user authentication, identity creation, and access control. It integrates SuperTokens for web authentication and Hyperledger Fabric CA for blockchain identity management, creating a bridge between the web application users and their blockchain identities.

```

async registerWithFabricCA(userId) {
  try {
    // Implementation
    console.log(`Successfully registered and enrolled user ${userId}
with Fabric CA`);

    return x509Identity;
  } catch (error) {
    console.error(`Failed to register user with Fabric CA: ${error}`);
    throw error;
  }
}

```

- **Blockchain Connection Management**

This layer handles the connection to the Hyperledger Fabric network, managing connection pooling, credential handling, and resource cleanup. It provides an abstraction over the low-level Fabric Gateway API and ensures efficient use of network resources.

```

async getUserConnection(userId, credentials = null) {
  // Create a new gRPC client connection
  client = await this.newGrpcConnection();

  try {
    // Implementation
    return contract;
  } catch (cryptoError) {
    return null;
  }
}

```

- **API Routing & Request Handling**

This layer defines the REST API endpoints and handles HTTP requests, translating them into blockchain operations. It implements the business logic for the voting application, including election management, voter registration, and vote casting.

```

app.route('/elections/:electionID/vote')
  .post(verifySession, async (req, res) => {
    try {
      const { candidateIndex, credentials } = req.body;
      const userId = req.session.getUserId();

      // Get user-specific connection using their credentials
      const contract = await this.connection.getUserConnection(userId,
credentials);

```

```

// Submit vote transaction to the blockchain
await contract.submitTransaction(
  'CastVote',
  req.params.electionID,
  userId,
  JSON.stringify(encryptedBallot),
  JSON.stringify(proofs)
);
res.status(200).send({ success: true, message: 'Vote cast
successfully' });
} catch (error) {
  console.error('Error casting vote:', error);
  res.status(500).send({ error: 'Failed to cast vote', details:
error.message });
}
});

```

- **Smart Contract (Chaincode) Layer**

This layer implements the blockchain smart contract (chaincode) that defines the data models, business rules, and transaction logic for the voting system. It ensures data integrity, enforces access control, and maintains the immutable record of elections and votes.

```

@Transaction(false)
@Returns('string')
public async GetElection(ctx: Context, electionID: string): Promise<string> {
  const electionKey = this.getElectionKey(electionID);
  const electionJSON = await ctx.stub.getState(electionKey);
  if (!electionJSON || electionJSON.length === 0) {
    throw new Error(`Election ${electionID} does not exist`);
  }
  return electionJSON.toString();
}

```

4.3 Ballot Encryption

```

const castVote = async (electionId, candidateId) => {
  const keyResponse = await api.get(`/publicKey`);
  const serializedPublicKey = keyResponse.data.publicKey;

  const { Voter, deserializePublicKey } = await import('mind-paillier-
voting-sdk');
  const publicKey = deserializePublicKey(serializedPublicKey)

  const voter = new Voter(1, publicKey);

```

```

const allProofs = [];
const encryptedBallot = [];

for (let i = 0; i < election.candidates.length; i++) {
  const bitValue = i === candidateIndex ? 1 : 0;
  const proofs = voter.encryptNumber(bitValue);
  encryptedBallot.push(proofs[0].c.toString());
  allProofs.push(proofs);
}

const proofsJson = JSON.stringify(allProofs, (key, value) =>
  typeof value === 'bigint' ? value.toString() : value
);
const response = await api.post(`/elections/${electionId}/vote`, {
  encryptedBallot: encryptedBallot,
  proofs: proofsJson,
  credentials: identityFile
});

```

Fetches public encryption key from the server and maps the chosen candidate to its index position within the election. Then creates a binary ballot (1 for selected candidate, 0 for others), and applies homomorphic encryption to each ballot position.

Lastly, creates zero-knowledge proofs proving vote validity without revealing choice, converts encrypted data to transmittable format, and sends encrypted ballot with proofs to blockchain network

4.4 Ballot Decryption

```

@Transaction(false)
@Returns('string')
public async GetElectionResults(ctx: Context, electionID: string,
privateKeyStr: string): Promise<string> {

  const publicKey = deserializePublicKey(keys.publicKey);
  const privateKey = deserializePrivateKey(privateKeyStr, publicKey);
  const tally: bigint[] = [];

  for (let i = 0; i < election.candidates.length; i++) {
    tally.push(publicKey.encrypt(BigInt(0)));
  }
  const votes = [];
  for await (const result of iterator) {
    if (result.value && result.value.toString()) {
      const vote = JSON.parse(result.value.toString()) as PrivateVote;

```

```

        votes.push(vote);
        for (let i = 0; i < vote.encryptedBallot.length; i++) {
            const encryptedValue = BigInt(vote.encryptedBallot[i]);
            if (i < tally.length) {
                tally[i] = publicKey.addition(tally[i], encryptedValue);
            }
        }
    }
}

// Decrypt the final tally
const decryptedResults = tally.map(encryptedSum => {
    const decrypted = privateKey.decrypt(encryptedSum);
    return Number(decrypted);
});
const finalResults = election.candidates.map((candidate, index) => ({
    candidateID: candidate.candidateID,
    name: candidate.name,
    party: candidate.party,
    voteCount: decryptedResults[index]
})));

```

When an election ends, the system retrieves all encrypted ballots from the blockchain without decrypting individual votes. By iteratively combining these encrypted ballots position by position, the system produces a single encrypted tally for each candidate. Only at the final stage is the private key used—just once—to decrypt these aggregated sums, revealing the total vote count for each candidate without ever exposing individual voter choices.

4.5 Project Installation and Execution

Make sure to have docker running for containerization of Hyperledger Fabric blockchain network.

Name	Date modified	Type	Size
 .git	25-05-2025 07:39 PM	File folder	
 backend	22-05-2025 08:31 PM	File folder	
 explorer	22-05-2025 08:31 PM	File folder	
 frontend	23-05-2025 03:40 AM	File folder	
 cloud-config.bash	22-05-2025 08:31 PM	Bash Source File	4 KB
 docker-compose.yaml	22-05-2025 08:31 PM	Yaml Source File	2 KB
 LICENSE.txt	22-05-2025 08:31 PM	Text Document	4 KB
 README.md	22-05-2025 08:31 PM	Markdown Source File	9 KB
 shutdown.sh	23-05-2025 02:55 AM	SH Source File	1 KB
 startup.sh	23-05-2025 02:54 AM	SH Source File	4 KB

Fig. 4.5 Project Directory Structure

On local server, the project components need to be started manually (on cloud, the cloud-config.bash handles the entire lifecycle).

- Install dependencies: ``npm install`` in both frontend and backend directories.
- Start Hyperledger Fabric network: run `startup.sh` script.
- Start backend:

```
cd backend/asset-transfer-basic/application-gateway-javascript  
npm start
```

- Start frontend:

```
cd frontend/  
npm run dev
```

- Access the app via the browser at the specified local address.

Chapter 5

Conclusion & Future Work

5.1 Conclusion

This project successfully demonstrated a secure, transparent, and privacy-preserving blockchain-based electronic voting platform. By integrating Hyperledger Fabric for ledger immutability, SuperTokens for robust authentication, a Node.js/Express.js backend, and a React.js frontend, the system offers a functional end-to-end voting solution. The incorporation of Homomorphic Encryption for tallying encrypted votes stands as a key achievement, significantly enhancing voter privacy without compromising the integrity of the election results.

The multi-tier, containerized architecture (using Docker) proved effective for modular development, deployment, and potential scalability. BlockVote effectively addresses many limitations of traditional and conventional e-voting systems by offering enhanced security, auditability, and user trust.

5.2 Future Work

While BlockVote establishes a strong foundation, several avenues exist for future enhancement and research:

1. **Advanced Cryptographic Protocols:** Explore more fully homomorphic encryption schemes to support complex tallying rules or richer ballot types.
2. **Scalability and Performance Optimization:** Investigate Hyperledger Fabric performance tuning, alternative consensus mechanisms, and off-chain computation techniques.

3. **Decentralized Identity (DID) Integration:** Explore integration with DID solutions for improved user-centric identity management and simplified, secure voter registration.
4. **Mobile Application Development:** Create native mobile applications (iOS/Android) to broaden accessibility and improve user convenience.
5. **Formal Security Audits:** Subject the system to rigorous third-party security audits and penetration testing to uncover and address potential vulnerabilities before any consideration for real-world deployment.

References

- [1] <https://www.ijraset.com/research-paper/public-perception-of-voting-systems-evaluating-trust>
- [2] <https://www.moneycontrol.com/msite/wazirx-cryptocontrol-articles/how-blockchain-can-facilitate-free-and-fair-elections-in-india/>
- [3] <https://electionbuddy.com/blog/2023/07/10/the-role-of-blockchain-in-electronic-voting-enhancing-security-traceability-and-trustworthiness/>
- [4] [Yan W. S., Silvia P. A., Salies A., Dinda R. H., Indrazno S., Mohammad S., Alfiandi A. R. \(November, 2024\). Blockchain-Based E-Voting System: A Decentralized Approach on the Ethereum Private Network. International Journal of Frontier Technology and Engineering](#)
- [5] <https://vaultody.com/blog/11-preserving-privacy-with-homomorphic-encryption>
- [6] <https://www.zama.ai/post/encrypted-onchain-voting-using-zk-and-fhe-with-zama-fhevm>
- [7] <https://hyperledger-fabric.readthedocs.io/en/release-2.5/>
- [8] <https://github.com/mind-network/mind-fhe-zk-voting-paillier-ts>
- [9] <https://darsh-patel.hashnode.dev/what-is-the-raft-consensus-algorithm-a-simple-guide>